35. Design LinkedIn People You May Know

Key Terms:

Term	Full Name	What it does
PYMK	People You May Know	LinkedIn's connection recommendation feature
FOF	Friends of Friends	2-hop connections in social graph
Graph DB	Graph Database	Database optimized for relationship queries
Jaccard Similarity	Set similarity metric	Overlap of two sets divided by union
Bloom Filter	Probabilistic data structure	Fast "already connected?" check
Feature Store	ML feature storage	Pre-computed features for model inference

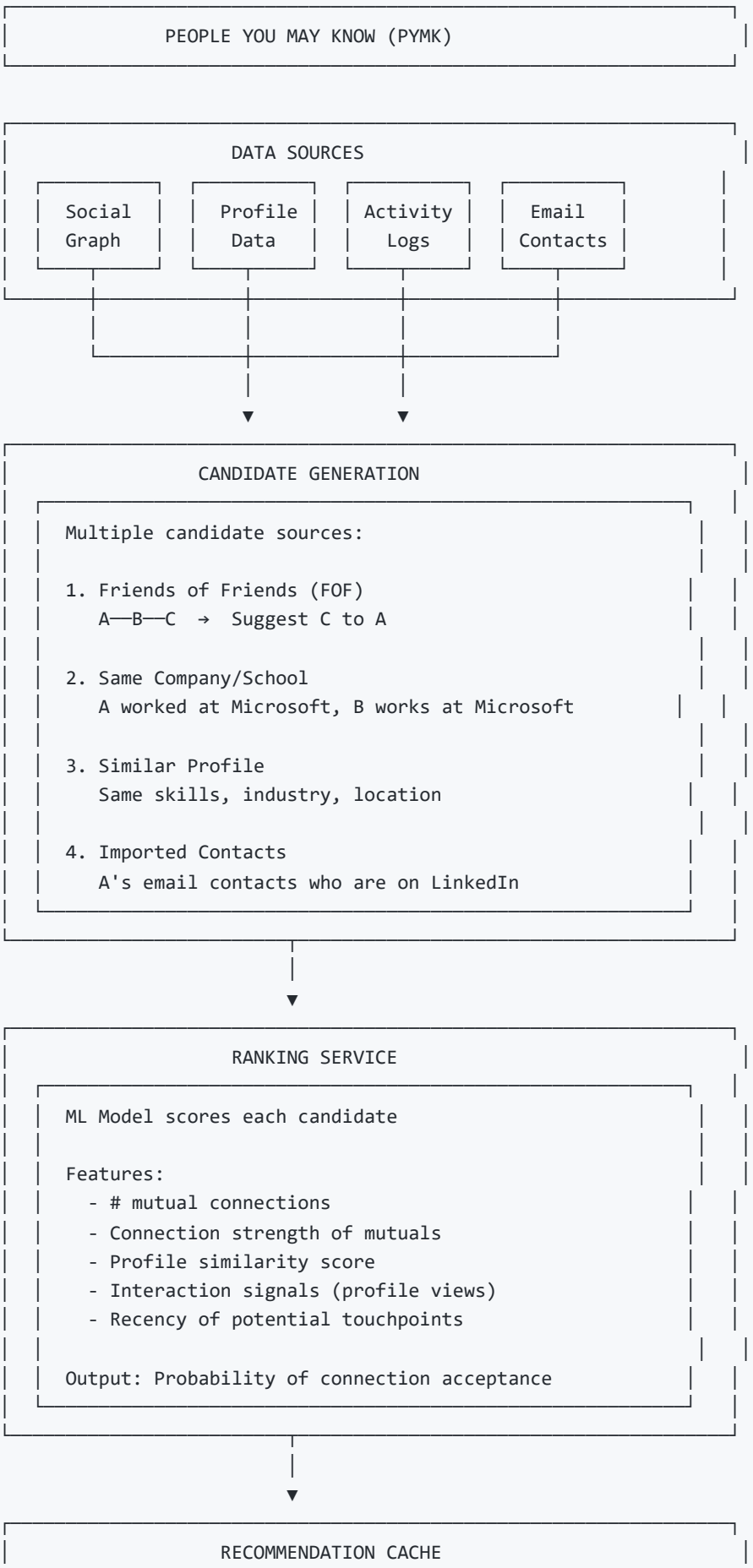
Problem Statement

Design a system that suggests relevant professional connections based on social graph analysis, shared attributes, and user behavior.

Clarifying Questions to Ask

1. **Scale**: Users, connections per user? (800M users, 500 avg connections)
2. **Update frequency**: How fresh should suggestions be?
3. **Signals**: What factors determine relevance?
4. **Privacy**: Respect user privacy settings?
5. **Diversity**: Mix of signal types in suggestions?
6. **Explanation**: Show why someone is recommended?
7. **Negative feedback**: Handle "Don't show" actions?

High-Level Design


PEOPLE YOU MAY KNOW (PYMK)
DATA SOURCES
Social Graph
Profile Data
Activity Logs
Email Contacts
CANDIDATE GENERATION
Multiple candidate sources:
1. Friends of Friends (FOF)
A—B—C → Suggest C to A
2. Same Company/School
A worked at Microsoft, B works at Microsoft
3. Similar Profile
Same skills, industry, location
4. Imported Contacts
A's email contacts who are on LinkedIn
RANKING SERVICE
ML Model scores each candidate
Features:
- # mutual connections
- Connection strength of mutuals
- Profile similarity score
- Interaction signals (profile views)
- Recency of potential touchpoints
Output: Probability of connection acceptance
RECOMMENDATION CACHE

```
│         Pre-computed top 100 suggestions per user                │
└──────────────────────────────────────────────────────────────────┘
```

API Design

Suggestion APIs

Method	Endpoint	Description
`GET`	`/api/v1/suggestions?userId={id}&limit=20`	Get "People You May Know" suggestions
`GET`	`/api/v1/suggestions/reasons/{suggestedUserId}`	Get why this person was suggested

Connection APIs

Method	Endpoint	Description
`POST`	`/api/v1/connections/request`	Send connection request
`PUT`	`/api/v1/connections/request/{requestId}/accept`	Accept request
`PUT`	`/api/v1/connections/request/{requestId}/reject`	Reject request
`DELETE`	`/api/v1/connections/{connectionId}`	Remove connection

Feedback APIs (to improve suggestions)

Method	Endpoint	Description
`POST`	`/api/v1/suggestions/{suggestedUserId}/dismiss`	Dismiss suggestion
`POST`	`/api/v1/suggestions/{suggestedUserId}/feedback`	Report irrelevant suggestion

Suggestion Response

```
GET /api/v1/suggestions?userId=user_123&limit=5

Response: 200 OK
{
  "suggestions": [
    {
```

```
        "user_id": "user_456",
        "name": "Jane Doe",
        "headline": "Senior Engineer at Microsoft",
        "mutual_connections": 12,
        "reasons": ["12 mutual connections", "Same company"],
        "score": 0.92
      }
    ]
  }
```

Key Components

Component	Purpose	Technology
Graph Database	Store social connections	Neo4j / JanusGraph
Profile Index	Search by attributes	Elasticsearch
Candidate Generator	Find potential connections	Batch Spark jobs
Ranking Model	Score candidates	TensorFlow / PyTorch
Cache	Store recommendations	Redis
Feedback Service	Handle dismissals	Event store

Graph-Based Candidate Generation

```
FRIENDS OF FRIENDS (FOF):
┌─────────────────────────────────────────────────────────────┐
│  Query: Find people within 2 hops who aren't connected      │
│                                                             │
│  MATCH (user:Person {id: $userId})                          │
│        -[:CONNECTED_TO]-(friend)                            │
│        -[:CONNECTED_TO]-(fof)                               │
│  WHERE NOT (user)-[:CONNECTED_TO]-(fof)                     │
│    AND fof <> user                                          │
│  RETURN fof, COUNT(friend) AS mutualCount                   │
│  ORDER BY mutualCount DESC                                  │
│  LIMIT 100                                                  │
│                                                             │
│  Result: People with most mutual connections               │
└─────────────────────────────────────────────────────────────┘

SCORING MUTUAL CONNECTIONS:
┌─────────────────────────────────────────────────────────────┐
│  Not all mutuals are equal:                                 │
│                                                             │
│  Strong mutual: Recent interaction, similar industry       │
```

```
│  Weak mutual: Connected 5 years ago, never interacted         │
│                                                               │
│  Weighted score = Σ (connection_strength[mutual])             │
└──────────────────────────────────────────────────────────────┘
```

Ranking Features

```
FEATURE CATEGORIES:

┌──────────────────────────────────────────────────────────────┐
│  GRAPH FEATURES:                                              │
│    - Mutual connection count                                  │
│    - Weighted mutual score                                    │
│    - Path length (2nd vs 3rd degree)                          │
│    - Common groups/companies                                  │
│                                                               │
│  PROFILE FEATURES:                                            │
│    - Industry match                                           │
│    - Location proximity                                       │
│    - Skill overlap (Jaccard similarity)                       │
│    - Seniority level match                                    │
│    - Company size similarity                                  │
│                                                               │
│  BEHAVIORAL FEATURES:                                         │
│    - Did user view candidate's profile?                       │
│    - Did candidate view user's profile?                       │
│    - Common content engagement                                │
│                                                               │
│  HISTORICAL FEATURES:                                         │
│    - Past connection acceptance rate (similar profiles)       │
│    - Time since candidate joined LinkedIn                     │
└──────────────────────────────────────────────────────────────┘
```

Scalability Considerations

- **Batch computation**: Pre-compute recommendations daily
- **Incremental updates**: Update when new connections made
- **Graph partitioning**: Shard graph by user geography
- **Bloom filters**: Quick "already connected?" check
- **Caching**: Store top 100 recommendations per user

Trade-offs

Decision	Option A	Option B	Recommendation
Computation	Real-time	Batch	**Batch + incremental**

Graph storage	SQL	Graph DB	**Graph DB** - FOF queries
Ranking	Rules	ML	**ML** - better accuracy
Explanation	Show	Hide	**Show** - builds trust

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Redis	Recommendation cache storage	Tech Guide
Elasticsearch	Profile search and attribute indexing	Tech Guide
Flink/Spark	Batch candidate generation	Tech Guide
Bloom Filter	Quick "already connected?" checks	Tech Guide

How to Explain It Well

> **Opening**: "PYMK uses graph analysis and ML ranking. Primary signal is mutual connections, enhanced with profile similarity and behavioral data."

> **Key talking points**:
>
> - "Friends-of-friends is primary source: 2-hop graph traversal"
> - "Weight mutual connections by interaction strength"
> - "ML model trained on historical accept/ignore actions"
> - "Batch compute daily, incremental update on new connections"
> - "Show explanation ('12 mutual connections') to build trust"
